SENAI “NADIR DIAS DE FIGUEIREDO”

Curso Técnico em Desenvolvimento de Sistemas



IKAI

Uso consciente de redes sociais e do tempo digital

RELATÓRIO TÉCNICO DE SOFTWARE

Sistema Web – Landing page de apresentação do projeto

Caio Soares Campos

Daniel Ribeiro Cruz

Felipe Brito das Chagas

Felipe Salvador Oliveira

Instrutores: Jansen Killinger Cara e Luiz Henrique

São Paulo – SP · Novembro de 2026 · Versão 1.0

Controle do documento

Tabela 1 – Histórico de revisões

Versão	Data	Descrição	Responsáveis
1.0	11/11/2026	Versão inicial do relatório técnico do Sistema Web, elaborada a partir da análise do repositório, do build de produção, de auditoria de dependências e de testes funcionais executados em navegador.	Equipe Ikai

Fonte: elaborado pelos autores a partir do código-fonte do projeto (2026).

Escopo deste documento

Este relatório trata **exclusivamente do Sistema Web** (landing page de apresentação do Ikai). O **Aplicativo Mobile** (Flutter) é documentado em relatório técnico próprio e aparece aqui apenas na medida em que se relaciona com o site.

Convenções de status

Implementado – presente no código e verificado em execução.

Parcial – existe, mas com efeito incompleto.

Não implementado – ausente ou apenas visual.

Planejado – previsto para a fase de distribuição do aplicativo.

Pendente de validação – não confirmável com o material disponível.

Sumário

1. Identificação do projeto 7
2. Resumo executivo 8
3. Introdução 9
3.1 Contextualização 9
3.2 Problema identificado 9
3.3 Justificativa 9
3.4 Objetivo geral 9
3.5 Objetivos específicos 9
3.6 Público-alvo 10
4. Visão geral da solução 11
4.1 Sistema Web 11
4.2 Aplicativo Mobile 12
4.3 Comunicação entre os componentes 12
4.4 Integrações e serviços externos 13
5. Levantamento e análise de requisitos 14
5.1 Requisitos funcionais (RF) 14
5.2 Requisitos não funcionais (RNF) 15
5.3 Regras de negócio 16
6. Arquitetura do software 17
6.1 Padrão arquitetural 17
6.2 Organização das camadas 17
6.3 Estrutura de diretórios 17
6.4 Responsabilidades dos principais módulos 18
6.5 Fluxo de comunicação entre as camadas 19
6.6 Comunicação entre Web, Mobile e Backend 19
6.7 Documentação da API (OpenAPI / Swagger) 19
7. Tecnologias e ferramentas utilizadas 22
7.1 Identidade visual 23
8. Modelagem e estrutura do banco de dados 24
8.1 Como os dados são tratados hoje 24
8.2 Limitações do modelo atual 24
8.3 Modelo proposto para a fase de distribuição 24
9. Funcionamento do sistema 26
9.1 Cabeçalho e navegação 26
9.2 Apresentação da proposta (hero) e simulação do aplicativo 26
9.3 Seções de conteúdo: manifesto, funcionamento, princípios e privacidade 26
9.4 Cadastro de interesse no lançamento 28
9.5 Botões de loja e rodapé 28
9.6 Responsividade 29
10. Diagramas e modelagem UML 31
10.1 Diagrama de casos de uso 31
10.2 Diagrama de atividades – cadastro de interesse 32
10.3 Diagrama de sequência – cadastro de interesse 32
10.4 Fluxograma de navegação da página 33
11. Segurança e privacidade 34

11.1 Mecanismos implementados e planejados 34
11.2 Avaliação 35
12. Testes e validação 36
12.1 Estratégia e evidências 36
12.2 Testes unitários 36
12.3 Testes funcionais executados no navegador 37
13. Resultados e limitações 39
13.1 Funcionalidades concluídas 39
13.2 Funcionalidades parciais ou pendentes 39
13.3 Resultados alcançados 39
13.4 Limitações técnicas identificadas 39
13.5 Problemas encontrados durante o desenvolvimento 40
14. Considerações finais e trabalhos futuros 42
14.1 Avaliação técnica 42
14.2 Trabalhos futuros 42
15. Referências e anexos 43
15.1 Documentação oficial e referências técnicas 43
15.2 Repositório 43
15.3 Anexos 44

Lista de figuras

Figura 1 – Visão geral da arquitetura e do fluxo de implantação do Sistema Web 11
Figura 2 – Seção inicial (hero) do site em resolução desktop 12
Figura 3 – Estrutura de diretórios do Sistema Web 17
Figura 4 – Árvore de componentes e elementos do Sistema Web 19
Figura 5 – Swagger UI – visão geral da documentação da integração 20
Figura 6 – Swagger UI – detalhe da operação POST /api/v1.0/email/send-form 21
Figura 7 – Fluxo de dados atual e modelo proposto para inscrições 25
Figura 8 – Seções “Como funciona” e “Princípios” 27
Figura 9 – Seção “Privacidade por princípio” 27
Figura 10 – Trecho da função submit() (App.jsx, identificadores abreviados) 28
Figura 11 – Seção Download com formulário de interesse e botões de loja 29
Figura 12 – Site em tablet e smartphone 30
Figura 13 – Diagrama de casos de uso do Sistema Web 31
Figura 14 – Diagrama de atividades do envio do formulário 32
Figura 15 – Diagrama de sequência do envio do formulário 32
Figura 16 – Mapa de seções e navegação por âncoras 33

Lista de tabelas

Tabela 1 – Histórico de revisões 2
Tabela 2 – Dados de identificação do projeto 7
Tabela 3 – Integrações e serviços externos 13
Tabela 4 – Requisitos funcionais do Sistema Web 14
Tabela 5 – Requisitos não funcionais 15
Tabela 6 – Regras de negócio 16
Tabela 7 – Camadas lógicas do Sistema Web 17
Tabela 8 – Responsabilidades dos módulos 18
Tabela 9 – Resumo da especificação OpenAPI 20
Tabela 10 – Tecnologias e ferramentas do Sistema Web 22
Tabela 11 – Paleta de cores (“wellness palette”) 23
Tabela 12 – Dados manipulados pelo Sistema Web 24
Tabela 13 – Modelo lógico proposto – entidade INSCRICAO_LANCAMENTO 25
Tabela 14 – Funcionalidade: cabeçalho e navegação 26
Tabela 15 – Funcionalidade: hero e simulação 26
Tabela 16 – Funcionalidade: seções de conteúdo 26
Tabela 17 – Funcionalidade: cadastro de interesse 28

Tabela 18 – Funcionalidade: botões de loja e rodapé 28
Tabela 19 – Comportamento responsivo 29
Tabela 20 – Segurança e privacidade: implementado × planejado 34
Tabela 21 – Tipos de teste e evidências 36
Tabela 22 – Testes unitários do Sistema Web 36
Tabela 23 – Casos de teste funcionais 37
Tabela 24 – Limitações técnicas do Sistema Web 39
Tabela 25 – Problemas encontrados e soluções 40
Tabela 26 – Roadmap de evolução do Sistema Web 42
Tabela 27 – Histórico de commits do repositório do site 43

1. Identificação do projeto

Tabela 2 – Dados de identificação do projeto

Campo	Descrição
Nome do projeto	Ikai
Componente documentado	Sistema Web – landing page de apresentação e pré-cadastro de interessados
Integrantes	Caio Soares Campos; Daniel Ribeiro Cruz; Felipe Brito das Chagas; Felipe Salvador Oliveira
Responsáveis pelo Sistema Web	Pelo histórico de versões, os 13 commits do repositório do site são de **Felipe Salvador Oliveira**. A participação dos demais integrantes (conteúdo, identidade visual, revisão) não aparece no histórico – **pendente de validação** pela equipe.
Instituição de ensino	SENAI “Nadir Dias de Figueiredo”
Curso	Técnico em Desenvolvimento de Sistemas
Orientação	Instrutores Jansen Killinger Cara e Luiz Henrique
Repositório	`github.com/felipesalvador-dev/Ikai---TCC` (pasta `ikai - frontend`)
Versão do documento	1.0
Data de elaboração	11 de novembro de 2026

Fonte: informações fornecidas pela equipe e histórico Git do repositório (2026).

Nomenclatura. O nome oficial adotado é **Ikai**, como aparece na interface. O `package.json` (`ikay-site`) e o README do site usam a grafia “Ikay”, tratada aqui como identificador legado a padronizar (Seção 13).

2. Resumo executivo

O **Ikai** é uma proposta de rede social voltada ao uso consciente do tempo digital: em vez de feed infinito e recomendações automáticas, **o usuário declara o que quer fazer e por quanto tempo, e o sistema respeita essa decisão**.

O **Sistema Web** é a vitrine pública do projeto. Trata-se de uma landing page de página única (SPA), desenvolvida em **React 19** com empacotamento pelo **Vite 6**, que tem dois objetivos:

- **comunicar a proposta**, o funcionamento previsto e os princípios do produto (sem feed infinito, sem recomendações comportamentais, sem rankings, privacidade por princípio);
- **captar interessados** no lançamento do aplicativo por meio de um formulário de e-mail integrado ao serviço externo **EmailJS**, sem necessidade de backend próprio.

O site é estático, responsivo (três faixas de layout) e leve: o build de produção gera um pacote JavaScript de 75,1 kB compactado (gzip). Não armazena dados pessoais nem usa banco de dados – o e-mail informado é entregue diretamente à caixa de correio da equipe.

Os testes funcionais executados em navegador confirmaram a renderização das seis seções, a navegação por âncoras, a validação do e-mail antes do envio, o bloqueio dos botões de loja e a ausência de rolagem horizontal em 1024, 768 e 390 px. Durante a elaboração deste relatório foi identificado e **corrigido** um defeito que deixava a logo quebrada no build de produção. O site passou a contar com **18 testes unitários** (todos aprovados), dependências sem vulnerabilidades conhecidas e **documentação OpenAPI/Swagger** da integração com o EmailJS.

> **Mensagens-chave para a apresentação**
>
> 1. O site não é o produto: ele **apresenta a ideia** e mede o interesse do público antes do lançamento do aplicativo.
> 2. A escolha por React + Vite gera um site estático, rápido e de hospedagem simples.
> 3. O EmailJS permite captar contatos **sem servidor nem banco de dados**, adequado à fase de apresentação.
> 4. Medidas de segurança e armazenamento estruturado estão planejadas para quando o aplicativo for disponibilizado para download.

3. Introdução

3.1 Contextualização

Plataformas sociais contemporâneas são otimizadas para maximizar o tempo de permanência do usuário, com rolagem infinita, reprodução automática, recomendações baseadas em comportamento, notificações constantes e comparação social. A decisão sobre o que consumir passa do usuário para o algoritmo. O Ikai surge como resposta a esse cenário, e o site é o primeiro contato do público com essa proposta.

3.2 Problema identificado

No plano do produto, o usuário acessa uma rede social com uma intenção e é conduzido a um consumo prolongado e não planejado. No plano do site, o problema é **apresentar uma ideia ainda em desenvolvimento** de forma clara e atraente, permitindo que pessoas interessadas acompanhem o lançamento sem que a equipe precise manter um servidor.

3.3 Justificativa

Uma landing page é o meio mais econômico de validar o interesse do público, apoiar a apresentação do TCC e criar uma lista de contatos para o lançamento. Por ter caráter apenas de apresentação, a solução prioriza simplicidade, desempenho e identidade visual, e deixa para a fase de distribuição do aplicativo os requisitos de segurança e armazenamento que a coleta em maior escala exige.

3.4 Objetivo geral

Desenvolver um site responsivo que apresente o conceito, o funcionamento previsto e os princípios do Ikai e permita aos visitantes registrar interesse no lançamento do aplicativo.

3.5 Objetivos específicos

1. Apresentar a proposta de valor e os princípios do produto em uma página única com navegação por seções. (**atingido**)
2. Simular visualmente a experiência do aplicativo para facilitar a compreensão do conceito. (**atingido**)
3. Captar e-mails de interessados sem backend próprio, com retorno visual ao visitante. (**atingido**)
4. Garantir layout adequado em desktop, tablet e smartphone. (**atingido**)
5. Publicar o site em ambiente acessível ao público. (**pendente de validação** – provedor não identificado)

3.6 Público-alvo

- **Potenciais usuários do Ikai:** jovens e adultos que desejam mais controle sobre o tempo nas redes sociais, especialmente estudantes.
- **Avaliadores e banca do TCC:** o site serve de material de apoio à apresentação.
- **Interessados no lançamento:** visitantes que deixam o e-mail para serem avisados.

4. Visão geral da solução

O Sistema Web é composto pelo código-fonte React, pelo processo de build do Vite, pelos arquivos estáticos gerados e pelo serviço externo EmailJS. A figura a seguir mostra os componentes e o caminho do código até o navegador do visitante.

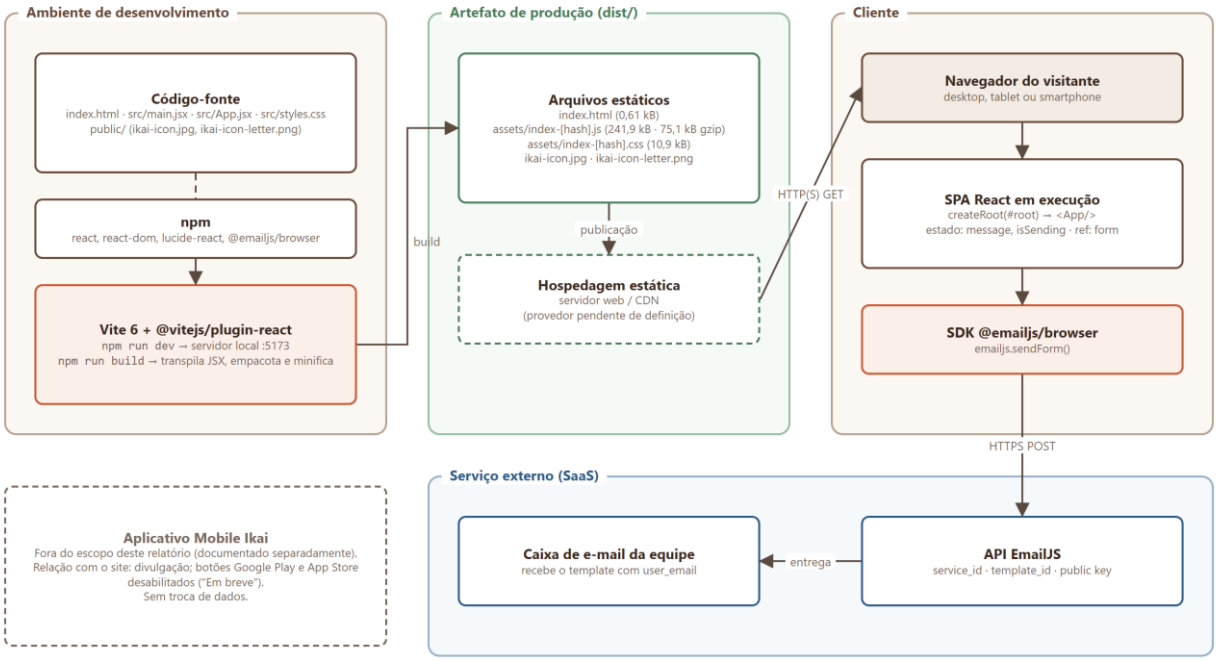


Figura 1 – Visão geral da arquitetura e do fluxo de implantação do Sistema Web

Fonte: elaborado pelos autores a partir do código-fonte do projeto (2026).

4.1 Sistema Web

Aplicação de página única com seis seções de conteúdo, cabeçalho fixo e rodapé. Não possui rotas, autenticação, backend nem banco de dados. Todo o conteúdo é declarado no próprio código; a única operação com efeito externo é o envio do formulário de interesse.

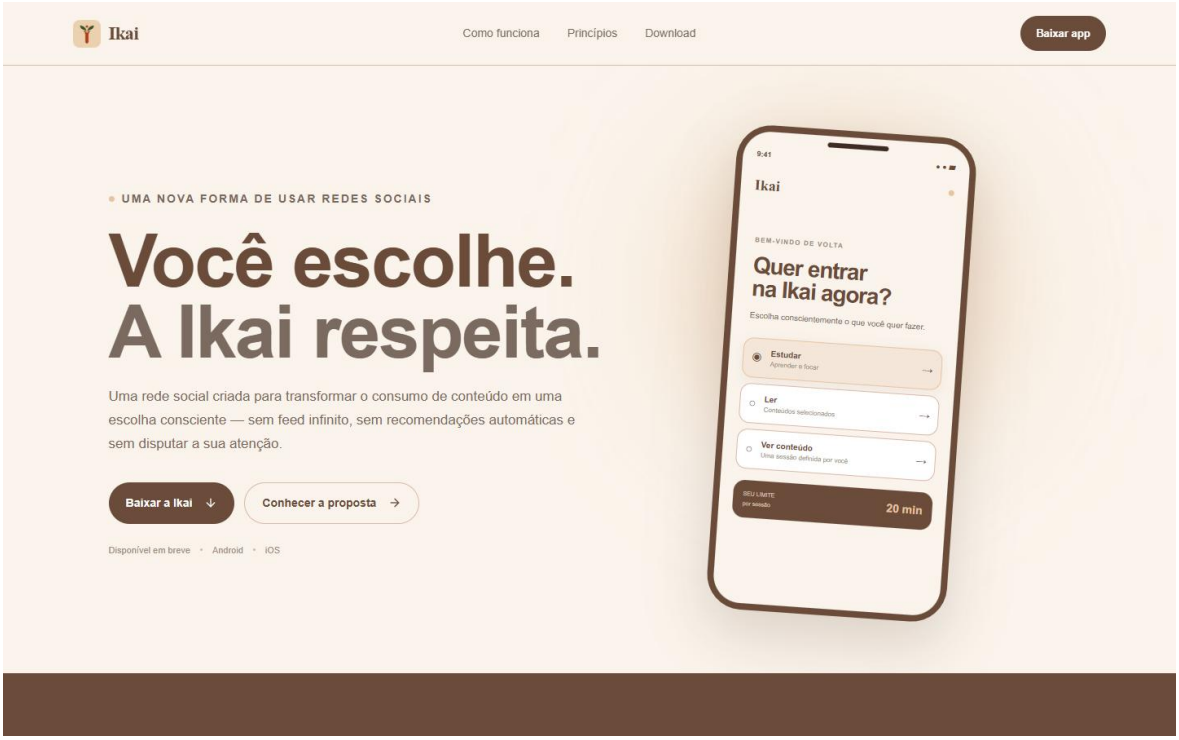


Figura 2 – Seção inicial (hero) do site em resolução desktop

Fonte: captura de tela do build de produção do site, 11/11/2026.

4.2 Aplicativo Mobile

O aplicativo Ikai (Flutter) está **fora do escopo deste relatório** e é documentado em relatório técnico próprio. A relação entre os dois componentes é exclusivamente de divulgação:

- o site descreve as funcionalidades previstas para o aplicativo (sessões conscientes, personalização manual, conteúdo finito, gamificação individual);
- o componente `PhonePreview` reproduz de forma estática a tela de escolha de objetivo do aplicativo;
- os botões “Google Play” e “App Store” estão desabilitados com o rótulo “Em breve”, aguardando a publicação.

4.3 Comunicação entre os componentes

Não há comunicação de dados entre o site e o aplicativo, nem backend compartilhado. No site, a comunicação ocorre em duas etapas: (1) o navegador baixa os arquivos estáticos do servidor de hospedagem; (2) o SDK do EmailJS envia, por HTTPS, o formulário à API do serviço, que entrega o e-mail à caixa da equipe.

4.4 Integrações e serviços externos

Tabela 3 – Integrações e serviços externos

Serviço / recurso	Uso no site	Situação
EmailJS (`@emailjs/browser` 4.4.1)	Envio do e-mail do formulário a um template configurado no painel do serviço.	**Implementado**
lucide-react 0.468	Ícones SVG empacotados no build (sem requisição externa).	**Implementado**
Fontes “DM Sans” e “Space Grotesk”	Declaradas no CSS, mas não importadas: o navegador usa a fonte sem serifa padrão.	**Parcial**
Lojas Google Play / App Store	Botões ilustrativos sem link.	**Planejado**
Ferramentas de análise de acesso (analytics)	Não utilizadas.	**Não implementado**

Fonte: elaborado pelos autores a partir do código-fonte do projeto (2026).

5. Levantamento e análise de requisitos

Os requisitos foram levantados a partir do código-fonte (`App.jsx`, `styles.css`, `index.html`), do README e do objetivo do site definido pela equipe (apresentação da ideia).

5.1 Requisitos funcionais (RF)

Tabela 4 – Requisitos funcionais do Sistema Web

ID	Requisito	Descrição / evidência	Status
RF01	Exibir cabeçalho fixo	`header.nav` com `position: sticky`, marca (logo + nome) e links de navegação.	**Implementado**
RF02	Navegar por âncoras	Links para `#como-funciona`, `#principios` e `#download`; marca aponta para `#inicio`; rolagem suave (`scroll-behavior: smooth`).	**Implementado**
RF03	Apresentar a proposta (hero)	Título “Você escolhe. A Ikai respeita.”, texto de apoio, CTAs “Baixar a Ikai” e “Conhecer a proposta” e indicação “Disponível em breve · Android · iOS”.	**Implementado**
RF04	Simular a interface do aplicativo	Componente `PhonePreview` com três escolhas (Estudar, Ler, Ver conteúdo) e limite “20 min por sessão”.	**Implementado**
RF05	Exibir manifesto	Seção “A ideia”.	**Implementado**
RF06	Descrever o funcionamento previsto	Seção “Como funciona” com seis cartões numerados (01–06).	**Implementado**
RF07	Apresentar os princípios	Seção “O princípio” com quatro itens e cartão de citação.	**Implementado**
RF08	Apresentar o compromisso de privacidade	Seção “Privacidade por princípio”.	**Implementado**
RF09	Cadastrar interesse no lançamento	Formulário com e-mail obrigatório, envio pelo EmailJS, estado “Enviando…”, mensagem de sucesso ou erro e limpeza do campo após sucesso.	**Implementado**
RF10	Indicar disponibilidade nas lojas	Botões Google Play e App Store com `type=button`, classe `disabled`, cursor `not-allowed` e rótulo “Em breve”.	**Implementado**
RF11	Exibir rodapé	Marca, slogan “Uma rede social feita para você.” e “© 2026 Ikai”.	**Implementado**
RF12	Adaptar o layout ao dispositivo	Media queries em 850 px e 560 px.	**Implementado**
RF13	Oferecer menu de navegação em telas pequenas	Abaixo de 850 px os links são ocultados e não há menu alternativo (ex.: menu hambúrguer).	**Não implementado**
RF14	Direcionar ao download nas lojas	Depende da publicação do aplicativo.	**Planejado**

Fonte: elaborado pelos autores a partir do código-fonte do projeto (2026).

5.2 Requisitos não funcionais (RNF)

Tabela 5 – Requisitos não funcionais

ID	Categoria	Requisito	Situação / evidência	Status
RNF01	Segurança	Trafegar o formulário por canal cifrado.	O SDK do EmailJS usa HTTPS. O HTTPS do próprio site depende da hospedagem.	Parcial
RNF02	Segurança	Proteger o formulário contra envio automatizado (spam).	Sem CAPTCHA nem limite próprio de envios.	Planejado
RNF03	Segurança	Manter dependências sem vulnerabilidades conhecidas.	`npm audit`: 0 vulnerabilidades após atualizar o Vite para 6.4.5 (versões fixadas e publicadas há mais de 7 dias).	Implementado
RNF04	Privacidade	Não armazenar dados pessoais no site.	Sem banco, cookies ou `localStorage`; o e-mail segue direto para o EmailJS.	Implementado
RNF05	Privacidade	Informar a finalidade da coleta e obter consentimento (LGPD).	O formulário diz “Cadastre-se para acompanhar o lançamento”, mas não há política de privacidade nem aceite explícito.	Parcial
RNF06	Desempenho	Carregar rapidamente.	JS 241,9 kB (75,1 kB gzip), CSS 10,9 kB, HTML 0,61 kB; carregamento local medido em 144 ms.	Implementado
RNF07	Usabilidade	Dar retorno claro às ações.	Botão “Enviando…”, campo desabilitado durante o envio, mensagens de sucesso e erro, efeitos de hover.	Implementado
RNF08	Acessibilidade	Ser utilizável por tecnologias assistivas.	`lang="pt-BR"`, rótulo oculto (`sr-only`) no campo, região `aria-live`, `alt` na logo. Ícones decorativos sem `aria-hidden`.	Parcial
RNF09	Disponibilidade	Estar acessível publicamente.	Site estático compatível com qualquer hospedagem; provedor e URL não identificados.	Pendente de validação
RNF10	Manutenibilidade	Facilitar alterações de conteúdo e estilo.	Conteúdo dos cartões em array (`features`); paleta em variáveis CSS; 18 testes unitários protegem contra regressões. Todo o código ainda está em um único componente de 556 linhas.	Parcial
RNF11	Compatibilidade	Funcionar em navegadores modernos e diferentes telas.	Build ES Modules; validado no Chrome em 1440, 1024, 768 e 390 px. Outros navegadores não testados.	Parcial
RNF12	Documentação	Documentar os contratos HTTP utilizados.	Especificação OpenAPI 3.0.3 da integração com o EmailJS em `swagger/openapi.yaml`, visualizável na Swagger UI (`npm run docs:api`).	Implementado

Fonte: elaborado pelos autores a partir do código-fonte do projeto (2026).

5.3 Regras de negócio

Tabela 6 – Regras de negócio

ID	Regra	Implementação
RN01	Só é possível enviar o formulário com um e-mail preenchido e em formato válido.	`<input type="email" required>` – validação nativa do navegador antes de `onSubmit`.
RN02	Não pode haver dois envios simultâneos.	`if (isSending) return;` e atributo `disabled` no campo e no botão enquanto `isSending` for verdadeiro.
RN03	Após o envio bem-sucedido, o visitante é informado e o formulário é limpo.	`setMessage("Cadastro realizado! …")` e `form.current.reset()`.
RN04	Em caso de falha, o visitante é informado e o erro é registrado.	Bloco `catch`: `console.error` e mensagem `Erro {status}: {text}`.
RN05	O estado de envio é sempre liberado ao final.	Bloco `finally`: `setIsSending(false)`.
RN06	O download só é oferecido após a publicação do aplicativo.	Botões de loja sem ação e com rótulo “Em breve”.
RN07	O site não coleta dados além do e-mail informado voluntariamente.	Ausência de cookies, analytics e armazenamento local.

Fonte: elaborado pelos autores a partir do código-fonte do projeto (2026).

Os princípios descritos no site (notificações desativadas por padrão, ausência de recomendações comportamentais, ausência de rankings e controle dos dados pelo usuário) são **diretrizes do produto Ikai**. Sua implementação pertence ao aplicativo e é avaliada no relatório próprio.

6. Arquitetura do software

6.1 Padrão arquitetural

O site é uma **aplicação de página única (SPA) baseada em componentes React**, renderizada inteiramente no cliente e distribuída como arquivos estáticos. Padrões como MVC ou Clean Architecture **não** são atribuídos ao projeto, pois não há separação entre modelo, controlador e serviços: dados, comportamento e marcação estão no componente `App`.

Justificativa técnica. Para uma página institucional com uma única interação externa, a composição por componentes funcionais com hooks (`useState`, `useRef`) é suficiente e dispensa bibliotecas de roteamento e de gerenciamento de estado. O Vite oferece servidor de desenvolvimento com recarga instantânea e build otimizado, e a saída estática pode ser hospedada sem servidor de aplicação.

6.2 Organização das camadas

Tabela 7 – Camadas lógicas do Sistema Web

Camada lógica	Responsabilidade	Artefatos
Documento base	Estrutura HTML, idioma, metadados, favicon e ponto de montagem.	`index.html` (`<div id="root">`)
Inicialização	Criar a raiz React e ativar o `StrictMode`.	`src/main.jsx`
Apresentação	Marcação JSX das seções e componentes reutilizáveis.	`App`, `Brand`, `Eyebrow`, `PhonePreview` (`src/App.jsx`)
Dados de conteúdo	Textos e ícones dos cartões e listas.	Array `features`, `choices` e lista de princípios
Comportamento	Estado do formulário e fluxo de envio.	`useState` (`message`, `isSending`), `useRef` (`form`), função `submit()`
Integração	Comunicação com o serviço externo.	`emailjs.sendForm` (`@emailjs/browser`)
Estilo	Identidade visual, layout e responsividade.	`src/styles.css` (986 linhas, variáveis CSS, grid/flexbox, 2 media queries)
Build	Transpilação de JSX, empacotamento e minificação.	Vite + `@vitejs/plugin-react` (`vite.config.js`)

Fonte: elaborado pelos autores a partir do código-fonte do projeto (2026).

6.3 Estrutura de diretórios

Figura 3 – Estrutura de diretórios do Sistema Web

```
ikai - frontend/
├── index.html            documento base (lang pt-BR, meta description, favicon)
├── vite.config.js        configuração do Vite (plugin React) e do Vitest (jsdom)
├── package.json          dependências e scripts: dev | build | preview | test | docs:api
├── package-lock.json     versões travadas das dependências
├── .gitignore            ignora node_modules
├── README.md             instruções de execução
├── public/               arquivos copiados sem processamento para a raiz do build
│   ├── ikai-icon.jpg          logo do cabeçalho e do rodapé
│   ├── ikai-icon-letter.png   favicon
│   └── icon-sem-fundo.png     variante da logo (não referenciada)
├── src/
│   ├── main.jsx          ponto de entrada
│   ├── App.jsx           componentes, conteúdo e integração com EmailJS (556 linhas)
│   ├── styles.css        estilos e responsividade (986 linhas)
│   ├── App.test.jsx      18 testes unitários (Vitest + Testing Library)
│   └── test/setup.js     configuração dos testes (jest-dom, limpeza entre testes)
├── swagger/              documentação OpenAPI/Swagger
│   ├── openapi.yaml          contrato da chamada à API do EmailJS
│   └── index.html            Swagger UI
└── dist/                 saída do build de produção (gerada por npm run build)
```

6.4 Responsabilidades dos principais módulos

Tabela 8 – Responsabilidades dos módulos

Módulo	Responsabilidade
`App` (App.jsx)	Componente raiz: compõe cabeçalho, seis seções e rodapé; mantém o estado do formulário e implementa `submit()`.
`Brand`	Logo e nome “Ikai” com link para o início; reutilizado no cabeçalho e no rodapé. Usa `import.meta.env.BASE_URL` para resolver o caminho da imagem.
`Eyebrow`	Rótulo de seção com marcador decorativo; recebe o texto por `children`.
`PhonePreview`	Moldura de smartphone com a tela simulada do aplicativo (objetivos e limite de sessão).
`features`	Array com número, ícone, título e texto dos cartões 01 a 04; os cartões 05 e 06 são escritos diretamente no JSX.
`styles.css`	Paleta (`--ink`, `--accent`, `--paper` etc.), tipografia, componentes visuais (botões, cartões, telefone, formulário) e regras responsivas.

Fonte: elaborado pelos autores a partir do código-fonte do projeto (2026).

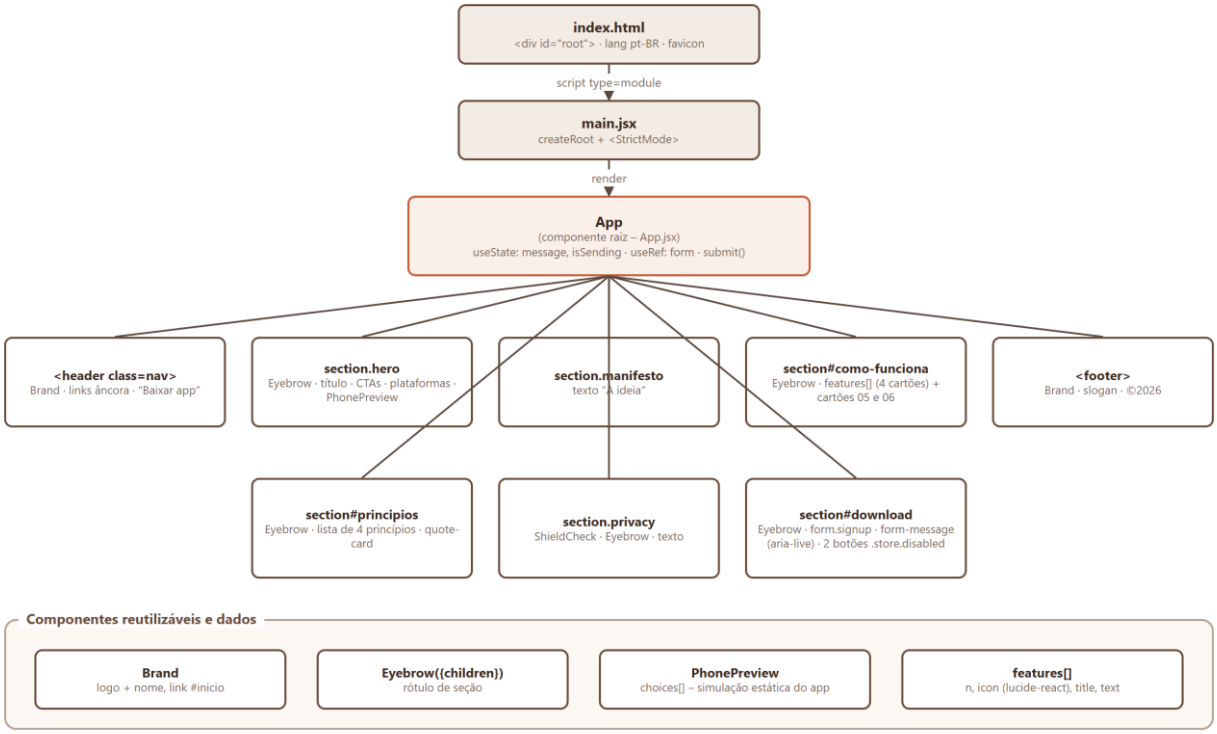


Figura 4 – Árvore de componentes e elementos do Sistema Web

Fonte: elaborado pelos autores a partir do código-fonte do projeto (2026).

6.5 Fluxo de comunicação entre as camadas

O fluxo é unidirecional, conforme o modelo do React: (1) o navegador carrega `index.html`, que importa `main.jsx`; (2) `createRoot` monta `<App/>` dentro de `#root`; (3) eventos do usuário (envio do formulário) chamam `submit()`; (4) `submit()` altera os estados `isSending` e `message` e chama o SDK do EmailJS; (5) cada alteração de estado provoca nova renderização do componente, refletindo "Enviando…" e a mensagem final. A navegação entre seções não passa pelo React: é feita pelo próprio navegador por meio das âncoras HTML.

6.6 Comunicação entre Web, Mobile e Backend

Não há backend próprio. O site não se comunica com o aplicativo móvel; a única comunicação de rede, além do carregamento da página, é a chamada HTTPS à API do EmailJS. Para a fase de distribuição do aplicativo, a Seção 8.3 propõe armazenar as inscrições de forma estruturada.

6.7 Documentação da API (OpenAPI / Swagger)

Como o site não possui API própria, a documentação Swagger descreve o **único contrato HTTP efetivamente utilizado**: a chamada que o SDK `@emailjs/browser` faz à API REST do EmailJS quando o formulário é enviado. A especificação foi escrita a partir do código do SDK instalado (`sendForm.js` e `sendPost.js`) e da documentação oficial do endpoint, sem incluir operações inexistentes.

Tabela 9 – Resumo da especificação OpenAPI

Item	Conteúdo
Arquivos	`swagger/openapi.yaml` (especificação OpenAPI 3.0.3) e `swagger/index.html` (Swagger UI 5.17.14)
Visualização	`npm run docs:api` → http://localhost:5174 (o botão "Try it out" foi desativado para impedir envios reais)
Servidor	`https://api.emailjs.com` (serviço externo)
Operação	`POST /api/v1.0/email/send-form` – `enviarInteresseLancamento`
Corpo	`multipart/form-data`: `user_email` (do formulário) + `lib_version`, `service_id`, `template_id` e `user_id` (acrescentados pelo SDK); `accessToken` opcional e não utilizado
Autenticação	Nenhum cabeçalho; a conta é identificada pela chave pública (`security: []`)
Respostas	`200 "OK"` (sucesso); `422` (exemplo oficial de falha); `4XX`/`5XX` com corpo textual, exibido como `Erro {status}: {texto}`
Erros locais do SDK	403, 429 e 451 (opções não configuradas no site) e `status 0 – Network Error` em falha de rede
Validação	`@redocly/cli lint`: especificação válida; único aviso é a ausência de licença, que o projeto não declara

Fonte: elaborado pelos autores a partir do código-fonte do projeto (2026).



Figura 5 – Swagger UI – visão geral da documentação da integração

Fonte: Swagger UI servida a partir de `swagger/openapi.yaml`, 11/11/2026.

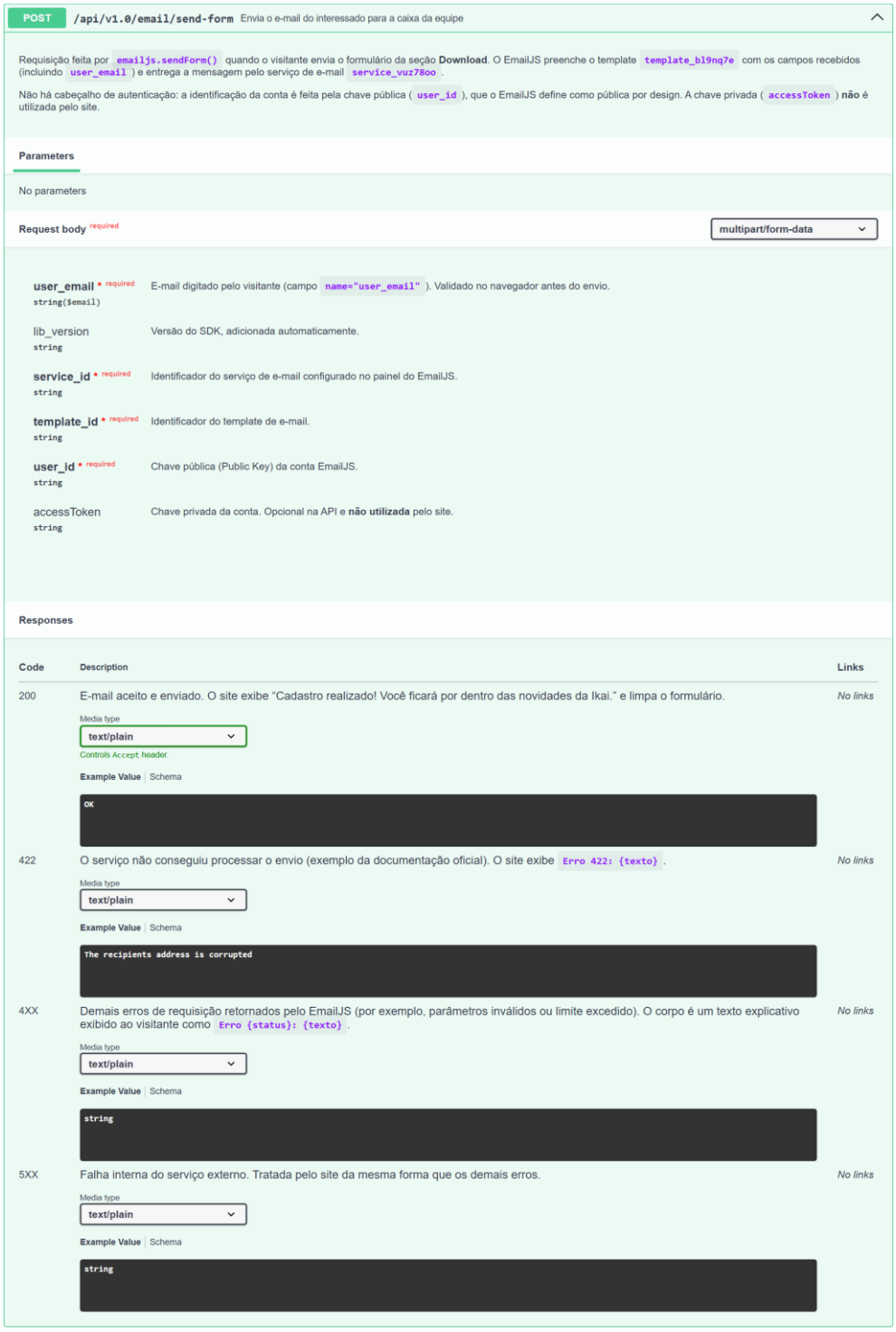


Figura 6 – Swagger UI – detalhe da operação POST /api/v1.0/email/send-form

Fonte: Swagger UI servida a partir de `swagger/openapi.yaml`, 11/11/2026.

7. Tecnologias e ferramentas utilizadas

Tabela 10 – Tecnologias e ferramentas do Sistema Web

Tecnologia / ferramenta	Categoria	Finalidade no projeto	Justificativa
HTML5	Linguagem de marcação	Documento base, semântica (`header`, `main`, `section`, `footer`) e validação nativa do formulário.	Padrão da web; a validação `type=email` dispensa código extra.
CSS3	Estilização	Variáveis, grid, flexbox, `clamp()`, `backdrop-filter`, transições e media queries.	Recursos nativos suficientes para uma única página, sem framework CSS.
JavaScript (ES Modules) + JSX	Linguagem	Lógica e componentes.	Linguagem nativa do navegador; JSX integra marcação e lógica.
React 19 (instalado 19.3.0)	Biblioteca de UI	Componentes funcionais e estado do formulário.	Modelo declarativo, ecossistema amplo e conhecimento da equipe.
Vite 6.4.5	Build e servidor de desenvolvimento	Recarga instantânea no desenvolvimento e build de produção otimizado.	Inicialização rápida e configuração mínima.
@vitejs/plugin-react (instalado 4.7.0)	Plugin de build	Transformação de JSX e Fast Refresh.	Plugin oficial do Vite para React.
lucide-react 0.468	Biblioteca de ícones	Ícones dos cartões, botões e seções.	SVG leves, importados individualmente.
@emailjs/browser 4.4.1	SDK de serviço externo	Envio do formulário de interesse.	Capta contatos sem backend nem banco de dados.
Vitest 4.1.12	Framework de testes	Execução dos testes unitários (`npm test`).	Integrado ao Vite: reutiliza a mesma configuração e transformação de JSX.
Testing Library (React 16.3, jest-dom 6.6, user-event 14.6)	Bibliotecas de teste	Renderização dos componentes, consultas por papel/rótulo e simulação de digitação e cliques.	Testa o comportamento como o usuário o percebe, sem depender de detalhes internos.
jsdom 26.1	Ambiente de teste	Simula o DOM do navegador no Node.js.	Permite testar o formulário, incluindo a validação nativa do e-mail, sem abrir um navegador.
OpenAPI 3.0.3 + Swagger UI 5.17	Documentação de API	Especificação e visualização da integração com o EmailJS.	Padrão de mercado para documentar contratos HTTP.
Redocly CLI 1.34	Validação de especificação	Verificação da especificação OpenAPI (`lint`).	Detecta erros de sintaxe e de estrutura no contrato.
Node.js + npm	Ambiente e gerenciador de pacotes	Instalação de dependências e execução dos scripts.	Padrão do ecossistema JavaScript.
Git + GitHub	Versionamento	Histórico e repositório remoto.	Rastreabilidade das alterações e backup.

Tecnologia / ferramenta	Categoria	Finalidade no projeto	Justificativa
Google Chrome (DevTools) + Puppeteer	Teste e depuração	Inspeção de layout, testes funcionais automatizados e capturas de tela.	Navegador de referência para validação.
Editor de código	Ferramenta de desenvolvimento	Edição do código-fonte.	Ferramenta específica utilizada pela equipe – pendente de validação.

Fonte: `package.json`, `package-lock.json` e `npm ls` (2026).

7.1 Identidade visual

Tabela 11 – Paleta de cores (“wellness palette”)

Variável CSS	Cor	Uso
`--ink`	#6B4C3B	Texto principal, botões primários, fundo do rodapé
`--accent`	#E8C5A0	Destaques, botão do formulário, mensagens
`--paper`	#FAF4EC	Fundo da página
`--soft`	#F5E6DA	Superfícies suaves
`--line`	#DDB8A8	Bordas e divisórias
`--muted`	#7A6A60	Textos secundários e ênfases
`--dark-card`	#594031	Cartões escuros e botões de loja

Fonte: `src/styles.css`.

A tipografia declarada é **Space Grotesk** para títulos e **DM Sans** para textos, mas essas fontes não são carregadas (Seção 13).

8. Modelagem e estrutura do banco de dados

> **Situação atual**
>
> **O Sistema Web não possui banco de dados.** Isso é coerente com seu objetivo de apresentação: o único dado coletado, o e-mail do interessado, é repassado ao EmailJS e entregue como mensagem à caixa de correio da equipe. O site não usa cookies, `localStorage` nem qualquer outro armazenamento.

8.1 Como os dados são tratados hoje

Tabela 12 – Dados manipulados pelo Sistema Web

Dado	Origem	Destino	Armazenamento
`user_email`	Campo do formulário (`name="user_email"`)	API do EmailJS → template de e-mail → caixa da equipe	Somente na caixa de e-mail e no histórico do EmailJS
`message`, `isSending`	Estado React	Interface	Memória do navegador (descartado ao recarregar)

Fonte: elaborado pelos autores a partir do código-fonte do projeto (2026).

8.2 Limitações do modelo atual

- Cada inscrição vira um e-mail isolado: não há consulta, contagem automática ou exportação estruturada.
- Não há deduplicação – o mesmo e-mail pode ser cadastrado várias vezes.
- Não há registro de consentimento nem mecanismo de descadastro, exigidos pela LGPD quando a lista for usada para comunicações.

8.3 Modelo proposto para a fase de distribuição

Para quando o aplicativo for disponibilizado, propõe-se armazenar as inscrições em uma entidade única. O modelo é apenas uma **proposta** e não está implementado.

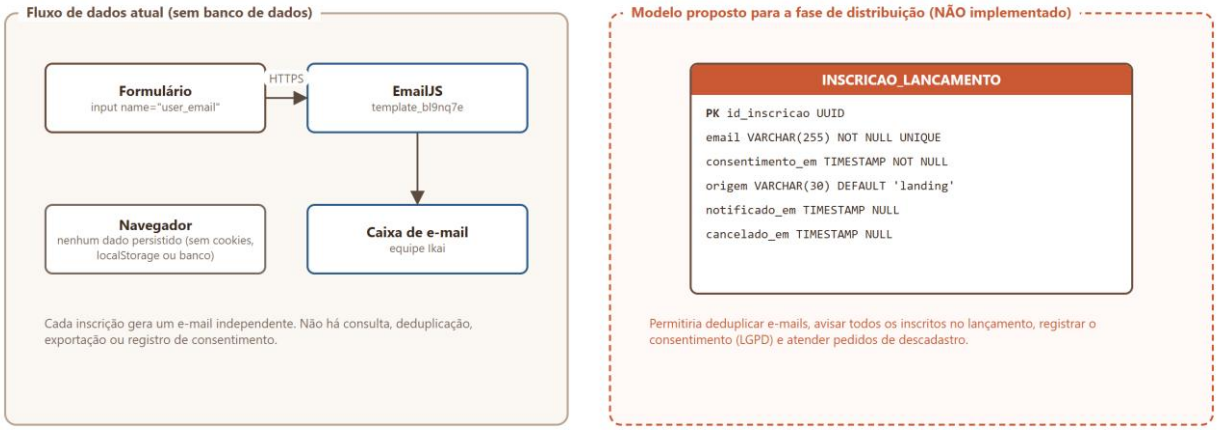


Figura 7 – Fluxo de dados atual e modelo proposto para inscrições

Fonte: elaborado pelos autores a partir do código-fonte do projeto (2026).

Tabela 13 – Modelo lógico proposto – entidade INSCRICAO_LANCAMENTO

Atributo	Tipo	Restrições	Finalidade
id_inscricao	UUID	Chave primária	Identificador único
email	VARCHAR(255)	NOT NULL, UNIQUE	Contato do interessado; a unicidade impede duplicidade
consentimento_em	TIMESTAMP	NOT NULL	Comprovação do aceite (LGPD)
origem	VARCHAR(30)	DEFAULT 'landing'	Canal de captação
notificado_em	TIMESTAMP	NULL	Data do aviso de lançamento
cancelado_em	TIMESTAMP	NULL	Descadastro; registros cancelados não recebem comunicações

Fonte: elaborado pelos autores a partir do código-fonte do projeto (2026).

Modelo conceitual: uma única entidade, sem relacionamentos, pois o visitante do site não possui conta. Quando o aplicativo tiver usuários cadastrados, a inscrição poderá ser associada à conta criada, conforme a modelagem descrita no relatório do aplicativo. A tecnologia de armazenamento (banco relacional, serviço gerenciado ou recurso de lista do próprio provedor de e-mail) é **pendente de definição**.

9. Funcionamento do sistema

Esta seção descreve cada funcionalidade do site. O funcionamento do aplicativo móvel é descrito em seu relatório próprio.

9.1 Cabeçalho e navegação

Tabela 14 – Funcionalidade: cabeçalho e navegação

Aspecto	Descrição
Objetivo	Permitir acesso rápido às seções principais a partir de qualquer ponto da página.
Funcionamento	Cabeçalho fixo no topo (78 px, fundo translúcido com desfoque) com a marca, três links (Como funciona, Princípios, Download) e o botão “Baixar app”. Os links usam âncoras; a rolagem é suave.
Dados envolvidos	Nenhum (links estáticos).
Fluxo de execução	Clique no link → navegador atualiza o `location.hash` → rolagem até a seção correspondente.
Resultado esperado	Seção de destino exibida no topo da janela. Em telas menores que 850 px os links são ocultados; abaixo de 560 px o botão “Baixar app” também.

Fonte: elaborado pelos autores a partir do código-fonte do projeto (2026).

9.2 Apresentação da proposta (hero) e simulação do aplicativo

Tabela 15 – Funcionalidade: hero e simulação

Aspecto	Descrição
Objetivo	Transmitir a proposta de valor em poucos segundos.
Funcionamento	Rótulo “Uma nova forma de usar redes sociais”, título “Você escolhe. A Ikai respeita.”, texto de apoio, dois CTAs (para `#download` e `#como-funciona`) e plataformas previstas. À direita, `PhonePreview` mostra a tela simulada “Quer entrar na Ikai agora?” com as opções Estudar (destacada), Ler e Ver conteúdo e o limite de 20 minutos.
Dados envolvidos	Array `choices` (título e descrição de cada opção).
Fluxo de execução	Renderização estática; os CTAs levam às seções correspondentes.
Resultado esperado	O visitante entende o conceito de sessão intencional antes de rolar a página.

Fonte: elaborado pelos autores a partir do código-fonte do projeto (2026).

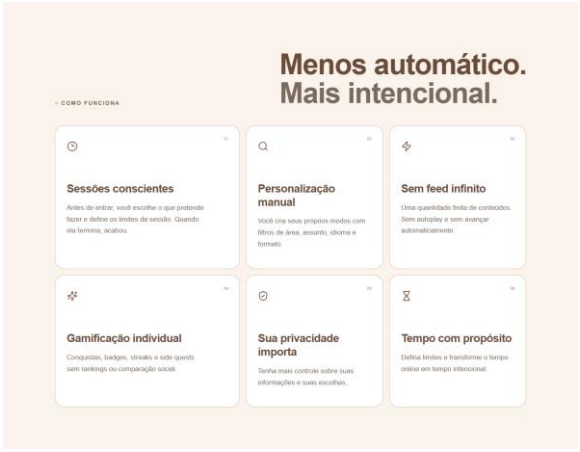
9.3 Seções de conteúdo: manifesto, funcionamento, princípios e privacidade

Tabela 16 – Funcionalidade: seções de conteúdo

Aspecto	Descrição
Objetivo	Explicar o que diferencia o Ikai de outras redes sociais.

Aspecto	Descrição
Funcionamento	**Manifesto:** “Uma rede social que não tenta prender você.” **Como funciona:** seis cartões – 01 Sessões conscientes, 02 Personalização manual, 03 Sem feed infinito, 04 Gamificação individual, 05 Sua privacidade importa, 06 Tempo com propósito. **Princípios:** “Low dopamine. Sem paternalismo.”, quatro itens (notificações desativadas por padrão; sem recomendações baseadas em comportamento; sem rankings; dados sob controle do usuário) e uma citação. **Privacidade:** “Seus dados não são um produto.” – armazenamento mínimo, compartilhamento iniciado pelo usuário, exportação e exclusão da conta.
Dados envolvidos	Array `features` (cartões 01–04) e conteúdo escrito no JSX.
Fluxo de execução	Renderização estática em grade responsiva (3, 2 ou 1 coluna conforme a largura).
Resultado esperado	Proposta compreendida; os itens descrevem o produto previsto e não funcionalidades do site.

Fonte: elaborado pelos autores a partir do código-fonte do projeto (2026).



Como funciona



Princípios

Figura 8 – Seções “Como funciona” e “Princípios”

Fonte: captura de tela do build de produção do site, 11/11/2026.



Figura 9 – Seção “Privacidade por princípio”

Fonte: captura de tela do build de produção do site, 11/11/2026.

9.4 Cadastro de interesse no lançamento

Tabela 17 – Funcionalidade: cadastro de interesse

Aspecto	Descrição
Objetivo	Registrar e-mails de pessoas que desejam ser avisadas do lançamento.
Funcionamento	O navegador valida o campo (`type=email`, `required`). Em seguida, `submit()` cancela o envio padrão, ativa `isSending` (campo e botão desabilitados, rótulo “Enviando…”) e chama `emailjs.sendForm` com o identificador do serviço, o identificador do template, o elemento do formulário e a chave pública.
Dados envolvidos	`user_email`.
Fluxo de execução	Ver diagramas de atividades e de sequência (Seções 10.2 e 10.3).
Resultado esperado	Sucesso: “Cadastro realizado! Você ficará por dentro das novidades da Ikai.” e campo limpo. Falha: mensagem com código e texto do erro e registro no console.

Fonte: elaborado pelos autores a partir do código-fonte do projeto (2026).

Figura 10 – Trecho da função submit() (App.jsx, identificadores abreviados)

```
async function submit(e) {
  e.preventDefault();
  if (isSending) return;                // RN02 – impede envio duplicado
  setIsSending(true);
  setMessage("");
  try {
    await emailjs.sendForm("service_…", "template_…", form.current, { publicKey: "…" });
    setMessage("Cadastro realizado! Você ficará por dentro das novidades da Ikai.");
    form.current.reset();               // RN03
  } catch (error) {
    console.error("EMAILJS ERROR:", error); // RN04
    setMessage(`Erro ${error?.status || ""}: ${error?.text || "Erro desconhecido no
EmailJS."}`);
  } finally {
    setIsSending(false);                // RN05
  }
}
```

9.5 Botões de loja e rodapé

Tabela 18 – Funcionalidade: botões de loja e rodapé

Aspecto	Descrição
Objetivo	Indicar onde o aplicativo estará disponível e encerrar a página com a identidade da marca.
Funcionamento	Dois botões (“Em breve na Google Play” e “Em breve na App Store”) com opacidade reduzida e cursor de bloqueio, sem ação. Rodapé com marca, slogan e direitos autorais.
Dados envolvidos	Nenhum.
Resultado esperado	O visitante entende que o download ainda não está disponível e é direcionado ao cadastro de interesse.

Fonte: elaborado pelos autores a partir do código-fonte do projeto (2026).



Figura 11 – Seção Download com formulário de interesse e botões de loja

Fonte: captura de tela do build de produção do site, 11/11/2026.

9.6 Responsividade

Tabela 19 – Comportamento responsivo

Faixa de largura	Comportamento do layout
Acima de 850 px	Hero em duas colunas (texto e telefone); cartões em 3 colunas; menu completo.
De 561 a 850 px	Hero em uma coluna; manifesto e princípios empilhados; cartões em 2 colunas; links do menu ocultos (permanece “Baixar app”).
Até 560 px	Título reduzido a 54 px; CTAs empilhados; telefone 270 × 545 px; cartões em 1 coluna; formulário e botões de loja empilhados; rodapé em coluna; “Baixar app” oculto.

Fonte: `src/styles.css`, media queries.



Tablet (768 px)



Smartphone (390 px)

Figura 12 – Site em tablet e smartphone

Fonte: captura de tela do build de produção do site, 11/11/2026.

10. Diagramas e modelagem UML

Os diagramas foram elaborados a partir do código-fonte e representam somente o comportamento implementado. O diagrama de arquitetura está na Seção 4 e a árvore de componentes, na Seção 6.4.

10.1 Diagrama de casos de uso

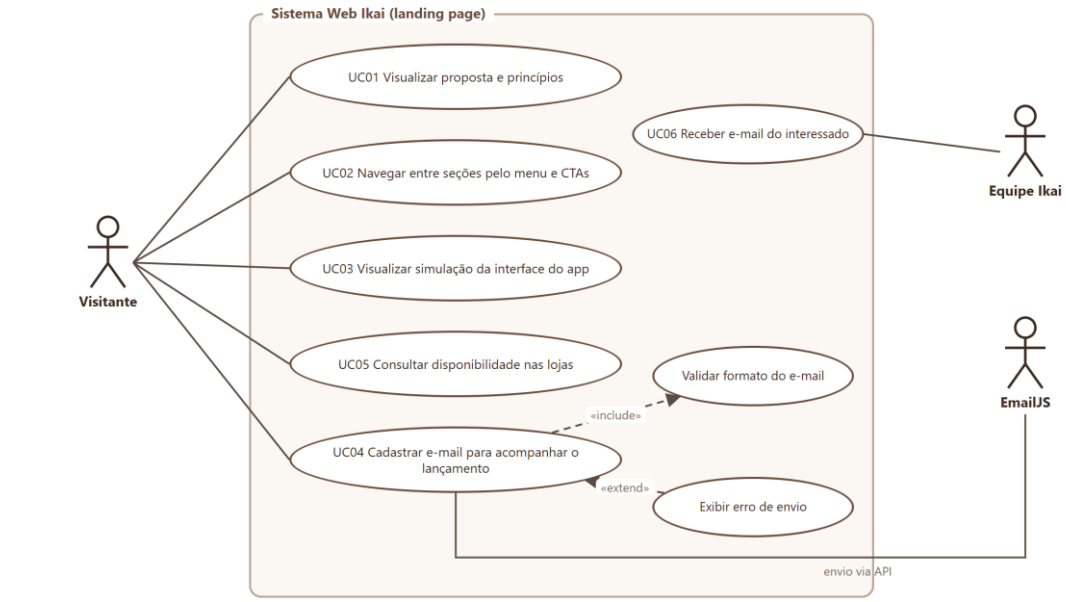


Figura 13 – Diagrama de casos de uso do Sistema Web

Fonte: elaborado pelos autores a partir do código-fonte do projeto (2026).

10.2 Diagrama de atividades – cadastro de interesse

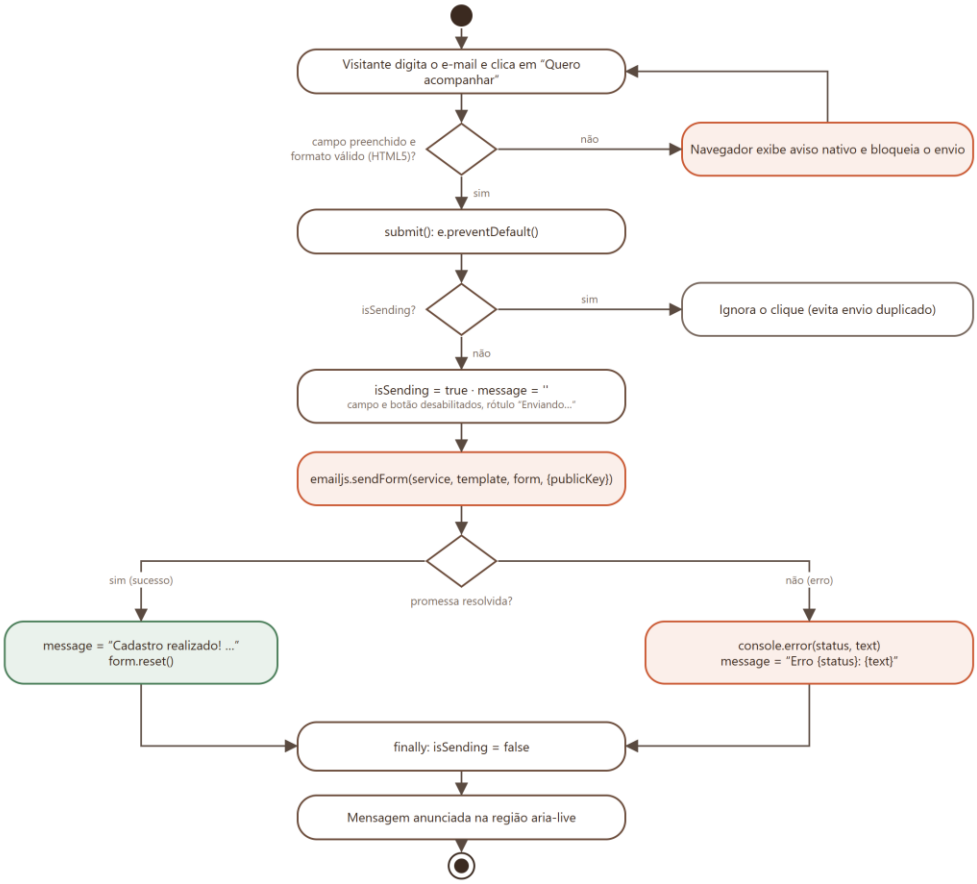


Figura 14 – Diagrama de atividades do envio do formulário

Fonte: elaborado pelos autores a partir do código-fonte do projeto (2026).

10.3 Diagrama de sequência – cadastro de interesse

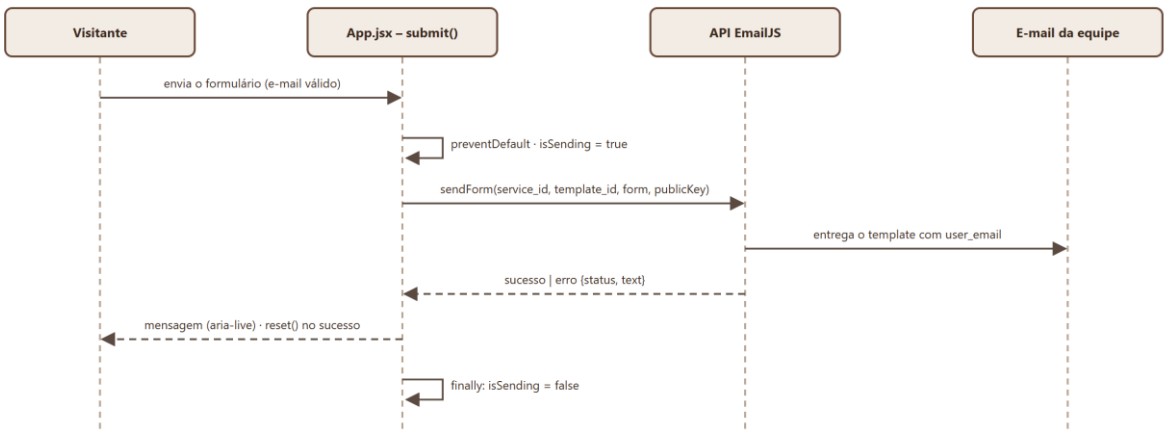


Figura 15 – Diagrama de sequência do envio do formulário

Fonte: elaborado pelos autores a partir do código-fonte do projeto (2026).

10.4 Fluxograma de navegação da página

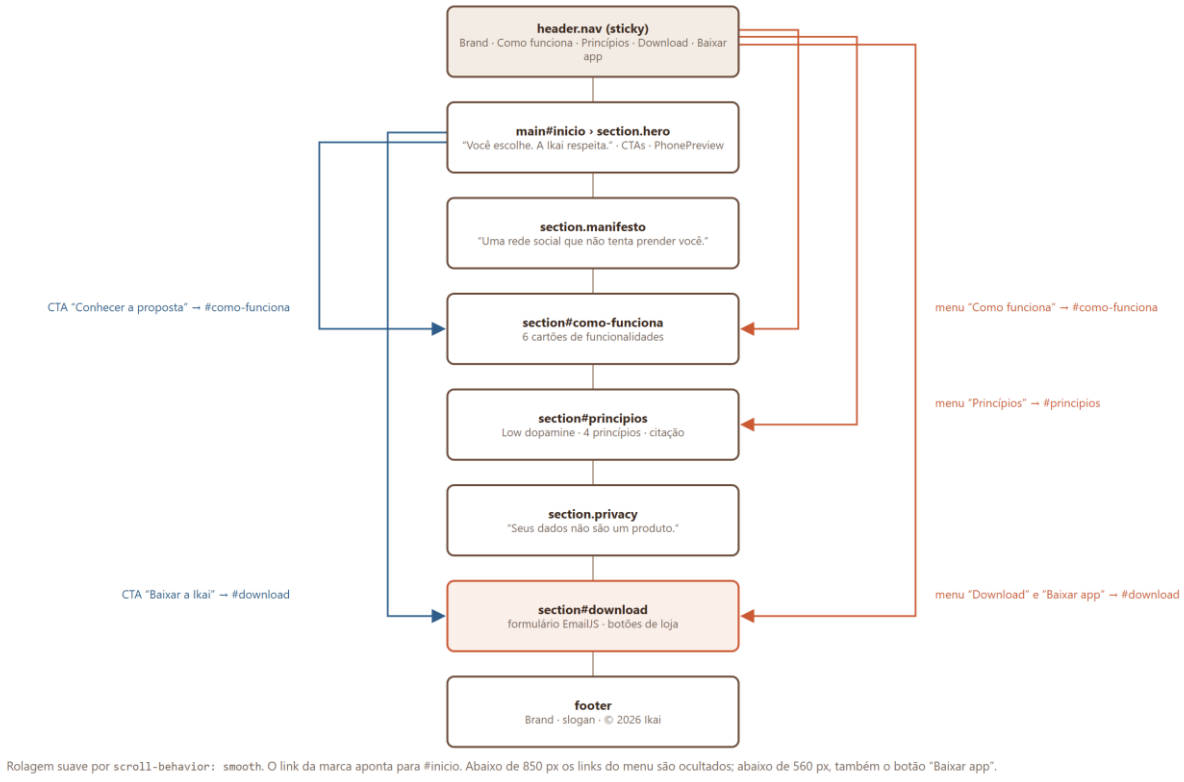


Figura 16 – Mapa de seções e navegação por âncoras

Fonte: elaborado pelos autores a partir do código-fonte do projeto (2026).

11. Segurança e privacidade

> **Escopo de segurança definido pela equipe**
>
> O Sistema Web tem caráter exclusivamente de **apresentação da ideia** e não armazena dados pessoais. Por decisão da equipe, as medidas de segurança adicionais serão implementadas na fase em que o **aplicativo for disponibilizado para download**. Esta seção separa o que já existe do que está planejado.

11.1 Mecanismos implementados e planejados

Tabela 20 – Segurança e privacidade: implementado × planejado

Aspecto	Situação atual	Status	Medida planejada
Autenticação e autorização	Não se aplicam: o site não possui área restrita nem contas.	N/A	—
Controle de acesso	Todo o conteúdo é público por definição.	N/A	—
Validação de informações	Validação nativa do e-mail (`type=email`, `required`) antes do envio; bloqueio de envio duplicado.	Implementado	Validação adicional no destino (ex.: confirmação por e-mail – double opt-in).
Proteção contra abuso	Sem CAPTCHA e sem limite de envios próprio.	Planejado	CAPTCHA no formulário; restrição de domínios autorizados e limites de envio no painel do EmailJS.
Credenciais do EmailJS	Identificadores do serviço e do template e a chave pública estão no código do cliente. A chave pública do EmailJS é pública por design; a chave privada não é usada.	Parcial	Mover para variáveis de ambiente do Vite (`VITE_*`) para facilitar a troca; ativar a lista de domínios permitidos.
Proteção de dados em trânsito	Envio ao EmailJS por HTTPS.	Implementado	Exigir HTTPS na hospedagem do site.
Armazenamento de informações	Nenhum dado é armazenado pelo site (sem banco, cookies ou `localStorage`).	Implementado	Lista estruturada com consentimento (Seção 8.3).
Tratamento de erros	Erros capturados em `try/catch`, registrados no console e exibidos ao visitante com o código e o texto técnico retornados pelo serviço.	Parcial	Exibir mensagem amigável e genérica; manter o detalhe apenas no console.
Dependências	`npm audit` com 0 vulnerabilidades: Vite atualizado de 6.4.3 para 6.4.5 e ferramentas de teste em versões sem alertas (Vitest 4.1.12). Versões fixadas e publicadas há mais de 7 dias.	Implementado	Repetir `npm audit` periodicamente.
Política de privacidade	O site descreve princípios de privacidade, mas não possui política formal nem aviso de finalidade junto ao formulário.	Planejado	Publicar a política de privacidade, indicar finalidade e base legal e oferecer canal de descadastro (LGPD).

Fonte: elaborado pelos autores a partir do código-fonte do projeto (2026).

11.2 Avaliação

Como nenhum dado pessoal é armazenado pelo site e o único dado coletado é fornecido voluntariamente para acompanhar o lançamento, o risco no estágio atual é baixo e compatível com o objetivo de apresentação. Antes de divulgar amplamente o formulário ou de usar a lista para comunicações, os itens marcados como **Planejado** – especialmente consentimento, política de privacidade e proteção contra spam – devem ser implementados.

12. Testes e validação

12.1 Estratégia e evidências

Tabela 21 – Tipos de teste e evidências

Tipo de teste	Situação	Evidência
Testes unitários	Executados	18 testes em `src/App.test.jsx` (Vitest + Testing Library + jsdom), todos aprovados – Seção 12.2.
Testes de integração	Não existem	—
Testes de API	Simulados	Não há API própria. A chamada à API do EmailJS é verificada nos testes unitários com o SDK simulado (`vi.mock`), e o contrato está documentado em OpenAPI (Seção 6.7).
Validação de banco de dados	Não aplicável	Não há banco de dados.
Build de produção	Executado	`npm run build` concluído sem erros em 1,9 s.
Auditoria de dependências	Executada	`npm audit`: 0 vulnerabilidades.
Testes funcionais e de interface	Executados	Roteiro abaixo, executado em 11/11/2026 no build de produção com Google Chrome automatizado (Puppeteer).

Fonte: elaborado pelos autores a partir do código-fonte do projeto (2026).

12.2 Testes unitários

Os testes unitários ficam em `src/App.test.jsx` e rodam com `npm test`. O SDK do EmailJS é **simulado** (`vi.mock("@emailjs/browser")`), o que permite verificar a chamada ao serviço, os estados de envio e o tratamento de sucesso e de erro sem enviar e-mails reais nem depender de rede. A interação é feita como um usuário faria (digitação e cliques com `user-event`).

Tabela 22 – Testes unitários do Sistema Web

#	Grupo	Teste	Status
1	Estrutura	Renderiza as seções principais com os ids usados pelas âncoras	**OK**
2	Estrutura	Aponta os links do menu para as seções corretas	**OK**
3	Estrutura	Direciona os CTAs do hero para download e para a proposta	**OK**
4	Estrutura	Carrega a logo da pasta public pela raiz do site (regressão do erro 404)	**OK**
5	Estrutura	Exibe os seis cartões de “Como funciona” na ordem	**OK**
6	Estrutura	Lista os quatro princípios do produto	**OK**
7	Estrutura	Mostra a simulação do app com as três escolhas e o limite de sessão	**OK**
8	Estrutura	Mantém os botões das lojas inativos até o lançamento	**OK**
9	Formulário	Exige um e-mail válido no campo (`type=email`, `required`, `name=user_email`)	**OK**
10	Formulário	Não envia quando o campo está vazio	**OK**

#	Grupo	Teste	Status
11	Formulário	Não envia quando o e-mail é inválido	OK
12	Formulário	Envia ao EmailJS com o serviço, o template e a chave pública	OK
13	Formulário	Mostra a mensagem de sucesso e limpa o campo	OK
14	Formulário	Desabilita campo e botão e mostra “Enviando...” durante o envio	OK
15	Formulário	Não dispara um segundo envio enquanto o primeiro está pendente	OK
16	Formulário	Mostra o erro do EmailJS, mantém o e-mail e reabilita o envio	OK
17	Formulário	Usa uma mensagem padrão quando o erro não traz detalhes	OK
18	Formulário	Anuncia a mensagem em uma região `aria-live`	OK

Fonte: saída de `npm test` (Vitest 4.1.12), 11/11/2026 – 18 aprovados em 4,6 s.

Verificação da eficácia dos testes. Para confirmar que os testes detectam falhas reais, três defeitos foram reintroduzidos temporariamente no código: o caminho antigo da logo (`./public/ikai-icon.jpg`), a remoção da trava `if (isSending) return` e a remoção do atributo `disabled` durante o envio. Os testes 4, 14 e 15 falharam como esperado, e o código foi restaurado em seguida.

12.3 Testes funcionais executados no navegador

Tabela 23 – Casos de teste funcionais

ID	Cenário	Resultado esperado	Resultado obtido	Status
CT01	Carregar a página	Título “Ikai — Use com intenção”, idioma pt-BR e 6 seções	Conforme esperado; nenhum erro no console	OK
CT02	Carregar a logo no build de produção	Imagem exibida no cabeçalho e no rodapé	Antes da correção: erro 404 em `/public/ikai-icon.jpg`. Depois: imagem carregada, sem requisições com falha	Corrigido
CT03	Clicar em “Princípios” no menu	Rolagem até `#principios`	`location.hash = #principios`; seção posicionada no topo	OK
CT04	Enviar o formulário vazio	Envio bloqueado com aviso	Aviso “Preencha este campo.”; nenhuma requisição ao EmailJS	OK
CT05	Enviar “usuario-sem-arroba”	Envio bloqueado com aviso de formato	Aviso “Inclua um ‘@’ no endereço de e-mail...”; nenhuma requisição	OK
CT06	Clicar nos botões de loja	Sem ação	Botões `type=button`, sem link nem manipulador de clique	OK
CT07	Largura de 1024 px	Menu visível, 3 colunas, sem rolagem horizontal	Conforme esperado	OK
CT08	Largura de 768 px	Links ocultos, 2 colunas, sem rolagem horizontal	Conforme esperado	OK

ID	Cenário	Resultado esperado	Resultado obtido	Status
CT09	Largura de 390 px	1 coluna, “Baixar app” oculto, sem rolagem horizontal	Conforme esperado	**OK**
CT10	Carregamento das fontes declaradas	DM Sans e Space Grotesk aplicadas	Nenhuma fonte web carregada (`document.fonts` vazio)	**Falhou**
CT11	Enviar e-mail válido	Mensagem de sucesso e e-mail recebido pela equipe	Não executado para evitar envio real ao EmailJS	**Não executado**

Fonte: execução automatizada no build de produção, 11/11/2026.

O caso CT11 deve ser executado pela equipe com um e-mail de teste, conferindo o recebimento na caixa configurada no EmailJS (**pendente de validação**). Recomenda-se também repetir os testes em Firefox, Safari e Edge e em pelo menos um smartphone físico.

13. Resultados e limitações

13.1 Funcionalidades concluídas

- Landing page completa com cabeçalho fixo, seis seções de conteúdo e rodapé.
- Simulação visual da experiência do aplicativo.
- Formulário de interesse integrado ao EmailJS, com validação, bloqueio de envio duplicado e retorno ao visitante.
- Layout responsivo em três faixas, sem rolagem horizontal.
- Build de produção leve e sem erros.
- 18 testes unitários aprovados, cobrindo estrutura, navegação e todo o fluxo do formulário.
- Documentação OpenAPI/Swagger da integração com o EmailJS.
- Dependências sem vulnerabilidades conhecidas (npm audit).

13.2 Funcionalidades parciais ou pendentes

- Navegação em telas pequenas: links ocultos sem menu alternativo.
- Tipografia: fontes declaradas, mas não carregadas.
- Download: botões de loja aguardam a publicação do aplicativo.
- Publicação: provedor de hospedagem e URL pública não identificados.

13.3 Resultados alcançados

O site cumpre seu papel de vitrine do projeto: apresenta a proposta com identidade visual consistente e funciona em diferentes tamanhos de tela. Ele também capta interessados sem infraestrutura de servidor, e serve como material de apoio para a apresentação ao comunicar visualmente o conceito de sessão intencional.

13.4 Limitações técnicas identificadas

Tabela 24 – Limitações técnicas do Sistema Web

ID	Limitação	Impacto	Recomendação
L01	Fontes “DM Sans” e “Space Grotesk” não importadas.	Tipografia diferente da planejada.	Importar pelo Google Fonts no `index.html` ou hospedar os arquivos.
L02	Sem menu de navegação abaixo de 850 px.	Em celulares só é possível rolar.	Implementar menu hambúrguer.
L03	O cabeçalho fixo (78 px) cobre o início da seção ao navegar por âncora.	Topo da seção fica sob o cabeçalho.	Adicionar `scroll-margin-top: 78px` às seções.
L04	Mensagem de erro exibe o texto técnico retornado pelo EmailJS.	Experiência confusa para o visitante.	Mensagem genérica ao usuário.

ID	Limitação	Impacto	Recomendação
L05	Sem CAPTCHA nem política de privacidade junto ao formulário.	Risco de spam; adequação à LGPD incompleta.	Ver Seção 11.
L06	Código morto: propriedade `feature.large` nunca definida (bloco `mini-ui` nunca exibido) e ícones `Hourglass`/`LucideHourglass` importados sem uso; `icon-sem-fundo.png` não referenciado.	Manutenibilidade.	Remover os trechos não utilizados.
L07	Todo o site em um único componente de 556 linhas.	Dificulta a manutenção à medida que o site crescer.	Separar seções em componentes próprios.
L08	`vite` e `@vitejs/plugin-react` listados em `dependencies` (as ferramentas de teste já estão em `devDependencies`).	Organização das dependências.	Mover para `devDependencies`.
L09	Sem linter configurado; testes não executados automaticamente a cada commit (sem CI).	Problemas de estilo e regressões dependem de execução manual de `npm test`.	Adicionar ESLint e um fluxo de CI (ex.: GitHub Actions) com `npm test` e `npm run build`.
L10	Grafia “Ikay” no `package.json` e no README; erro de digitação “dizer a o Ikai” na citação.	Consistência da marca.	Padronizar para “Ikai” e corrigir o texto.
L11	Sem metadados de compartilhamento (Open Graph) e sem favicon em múltiplos tamanhos.	Prévia pobre ao compartilhar o link.	Adicionar `og:title`, `og:image` etc.
L12	Pasta `dist/` não está no `.gitignore`.	Risco de versionar arquivos gerados.	Incluir `dist` no `.gitignore`.

Fonte: elaborado pelos autores a partir do código-fonte do projeto (2026).

13.5 Problemas encontrados durante o desenvolvimento

Tabela 25 – Problemas encontrados e soluções

Problema	Evidência	Solução
Pastas `node_modules` e `dist` versionadas no repositório.	Commits “acrescentando .gitignore”, “deletando node_modules” e “removendo dist” (16/10/2026).	Criação do `.gitignore` e remoção das pastas do repositório.
Ícone da aba (favicon) incorreto.	Commits “atualizando a logo da guia” (14/10) e “arrumando a logo da guia” (16/10).	Ajuste do arquivo e do caminho do favicon.
Logo quebrada no build de produção: caminho `./public/ikai-icon.jpg` só funcionava no servidor de desenvolvimento, porque o Vite publica o conteúdo de `public/` na raiz do site.	Erro 404 no teste CT02.	**Corrigido nesta versão:** a logo passou a usar `import.meta.env.BASE_URL` + `ikai-icon.jpg` e o favicon passou a apontar para `/ikai-icon-letter.png`.
Dependências do Vite com alertas de segurança (`npm audit`).	2 alertas altos no Vite 6.4.3.	Atualização para Vite 6.4.5; `npm audit` com 0 vulnerabilidades.

Problema	Evidência	Solução
Ajustes de tamanho de fonte e espaçamento.	Commits “ajustando o tamanho das letras” (23/10) e “ajustando css” (28/10).	Refinamento do `styles.css`.

Fonte: elaborado pelos autores a partir do código-fonte do projeto (2026).

14. Considerações finais e trabalhos futuros

14.1 Avaliação técnica

O Sistema Web do Ikai atinge o objetivo para o qual foi concebido: apresentar a ideia de forma clara, com identidade visual coerente com a proposta de bem-estar digital, e captar interessados sem exigir infraestrutura de servidor. As escolhas tecnológicas – React com Vite, CSS nativo e EmailJS – são proporcionais ao escopo e resultam em um site leve, rápido e simples de publicar. O comportamento do formulário é protegido por 18 testes unitários, e a integração externa está documentada em OpenAPI/Swagger.

As limitações identificadas são, em sua maioria, de acabamento (tipografia, menu móvel, mensagens de erro) e de preparação para escala (consentimento, proteção contra spam, armazenamento estruturado das inscrições). Nenhuma delas compromete a função de apresentação, e todas têm solução direta descrita neste documento.

14.2 Trabalhos futuros

Tabela 26 – Roadmap de evolução do Sistema Web

Prioridade	Ação	Referência
Alta	Executar o teste de envio real do formulário (CT11) e publicar o site com HTTPS.	CT11, RNF09
Alta	Importar as fontes, implementar o menu móvel e ajustar `scroll-margin-top`.	L01, L02, L03
Alta (fase de download)	Política de privacidade, consentimento no formulário, CAPTCHA e restrição de domínio no EmailJS.	Seção 11
Alta (fase de download)	Ativar os botões das lojas com os links reais do aplicativo.	RF14
Média	Armazenar as inscrições de forma estruturada (deduplicação, aviso de lançamento, descadastro).	Seção 8.3
Média	Mover `vite` e `@vitejs/plugin-react` para `devDependencies`.	L08
Média	Separar as seções em componentes e remover código morto.	L06, L07
Baixa	ESLint, CI com testes automáticos, metadados Open Graph e padronização do nome.	L09, L10, L11

Fonte: elaborado pelos autores a partir do código-fonte do projeto (2026).

15. Referências e anexos

15.1 Documentação oficial e referências técnicas

BRASIL. Lei nº 13.709, de 14 de agosto de 2018. Lei Geral de Proteção de Dados Pessoais (LGPD). Disponível em: https://www.planalto.gov.br/ccivil_03/_ato2015-2018/2018/lei/l13709.htm.

EMAILJS. EmailJS Documentation – Browser SDK. Disponível em: https://www.emailjs.com/docs/.

LUCIDE. Lucide React. Disponível em: https://lucide.dev/guide/packages/lucide-react.

MDN WEB DOCS. Client-side form validation. Disponível em: https://developer.mozilla.org/docs/Learn/Forms/Form_validation.

MDN WEB DOCS. ARIA live regions. Disponível em: https://developer.mozilla.org/docs/Web/Accessibility/ARIA/ARIA_Live_Regions.

OPENAPI INITIATIVE. OpenAPI Specification v3.0.3. Disponível em: https://spec.openapis.org/oas/v3.0.3.

EMAILJS. REST API – /send-form. Disponível em: https://www.emailjs.com/docs/rest-api/send-form/.

SMARTBEAR. Swagger UI. Disponível em: https://swagger.io/tools/swagger-ui/.

TESTING LIBRARY. React Testing Library. Disponível em: https://testing-library.com/docs/react-testing-library/intro/.

VITEST. Vitest Guide. Disponível em: https://vitest.dev/guide/.

NPM. npm-audit. Disponível em: https://docs.npmjs.com/cli/commands/npm-audit.

REACT. React documentation. Disponível em: https://react.dev.

VITE. Vite Guide – Static Asset Handling / The public directory. Disponível em: https://vite.dev/guide/assets.html.

15.2 Repositório

`https://github.com/felipesalvador-dev/Ikai---TCC` – pasta `ikai - frontend`.

Tabela 27 – Histórico de commits do repositório do site

Commit	Data	Mensagem
0c10c56	18/09/2026	first commit
9d52f1f	18/09/2026	first commit
c0d13e1	23/09/2026	second commit
5c16a9e	23/09/2026	third commit
0581cfa	14/10/2026	atualizando a logo da guia
c6f47e8	14/10/2026	alterações estéticas
a06b215	16/10/2026	acrescentando .gitignore

Commit	Data	Mensagem
d9c3530	16/10/2026	deletando node_modules
298304d	16/10/2026	removendo dist
1bedaa6	16/10/2026	arrumando a logo da guia
b7db961	16/10/2026	2 cards adicionados
de98266	23/10/2026	ajustando o tamanho das letras
751f762	28/10/2026	ajustando css

Fonte: `git log` do repositório, 11/11/2026.

15.3 Anexos

Anexo A – Comandos de execução

```
cd "ikai - frontend"
npm install          # instala as dependências
npm run dev          # servidor de desenvolvimento (http://localhost:5173)
npm run build        # gera a versão de produção em dist/
npm run preview      # pré-visualiza o build de produção
npm test             # executa os 18 testes unitários
npm run test:watch   # testes em modo observação
npm run docs:api     # Swagger UI em http://localhost:5174
```

Anexo B – Saída do build de produção (11/11/2026)

```
vite v6.4.5 building for production...
✓ 1599 modules transformed.
dist/index.html                  0.61 kB │ gzip:  0.36 kB
dist/assets/index-DkCAGKSU.css 10.91 kB │ gzip:  2.94 kB
dist/assets/index-CH2Xrnzc.js 241.88 kB │ gzip: 75.12 kB
✓ built in 1.96s
```

Anexo C – Saída dos testes unitários (11/11/2026)

```
 RUN  v4.1.12  ikai - frontend
 ✓ src/App.test.jsx (18 tests) 3.13s

 Test Files  1 passed (1)
      Tests  18 passed (18)
   Duration  4.61s
```

Anexo D – Evidências. As capturas de tela deste documento foram geradas em 11/11/2026 a partir do build de produção, servido localmente e acessado pelo Google Chrome nas larguras de 1440, 768 e 390 px.